



Ca' Foscari  
University  
of Venice

---

# Standard and metrics in Software Quality Assurance

Pietro Ferrara

[pietro.ferrara@unive.it](mailto:pietro.ferrara@unive.it)



Ca' Foscari  
University  
of Venice

---



sw bug



## WIRED

LILY HAY NEWMAN SECURITY 12.31.17 07:00 AM

# THE WORST HACKS OF 2017

### Equifax

This was really bad. The credit monitoring firm Equifax disclosed a massive breach at the beginning of September, which exposed personal information for 145.5 million people. The data included birth dates, addresses, some driver's license numbers, about 209,000 credit card numbers, and Social Security numbers—meaning that almost half the US population potentially had their crucial secret identifier exposed. Because the information Equifax coughed up was so sensitive, it's widely considered the worst corporate data breach ever. For now.



**2012**

Bug introduced in open source software Apache Struts

Find during development

**2012, 2015, 2016**

Several versions released

Find at release

**Between**

Several versions released

Find at release

**March 7, 2017**

Vulnerability discovered

Find when acquiring third-party software

**May 13, 2017**

Hackers access Equifax files



Ca' Foscari  
University  
of Venice

# Nearly 157,000 had data breached in TalkTalk cyber-attack

BBC Sign in

NEWS

Friday 6 November 2015 08.32 GMT

BUSINESS  
INSIDER  
UK

## The TalkTalk hack cost it £42 million

May 12, 2016, 8:33 AM

TalkTalk unveiled the full impact of the cyber attack it suffered last year — 95,000 customers left in the immediate aftermath and the cleanup cost hit £42 million (\$60.53 million).



theguardian

## TalkTalk hit with record £400k fine over cyber-attack

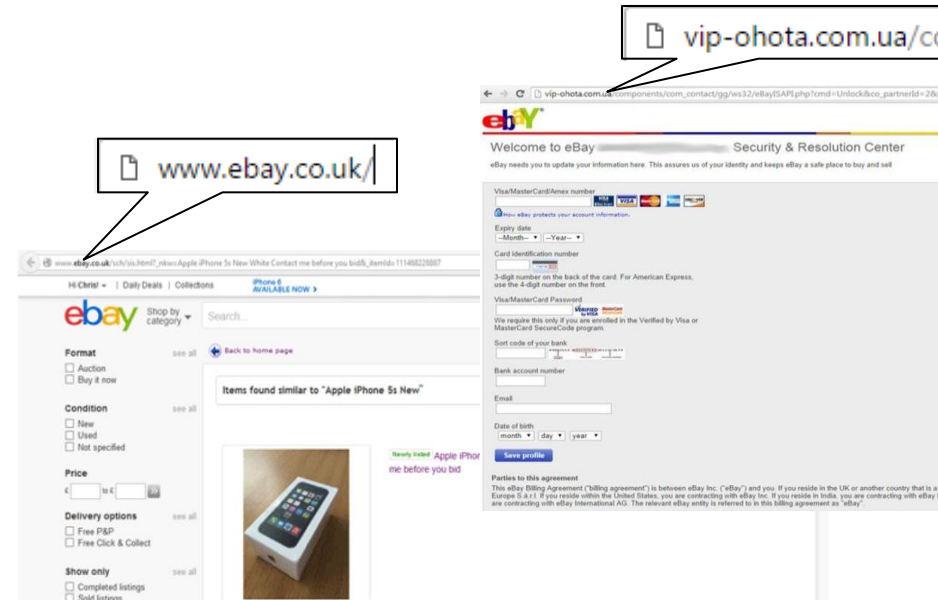
Wednesday 5 October 2016 14.00 BST

“The technique used by the attacker, called **SQL injection**, has been well known in security circles for almost 20 years. “SQL injection is well understood, defences exist and TalkTalk ought to have known it posed a risk to its data,” the Information Commissioner’s Office said.”



## What is SQL injection?

«permits the attacker to modify existing data, obtain system data, eliminate data or make it inaccessible»



## What is XSS?

«Cross-Site Scripting is a security vulnerability permitting the hacker to insert content on web pages, without the owner knowing it, and divert the user elsewhere»



Ca' Foscari  
University  
of Venice

BBC

Sign in

News

Sport

Weather

Shop

Earth

Travel

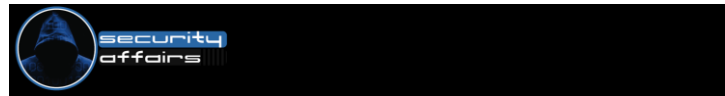
More

## NEWS

Home Video World UK Business Tech Science Magazine Entertainment & Arts

24 July 2015 | Technology

### Fiat Chrysler recalls 1.4 million cars after Jeep hack



Security vulnerabilities in the Hyundai Blue Link mobile apps allowed hackers to steal vehicles, the car maker fixed them.

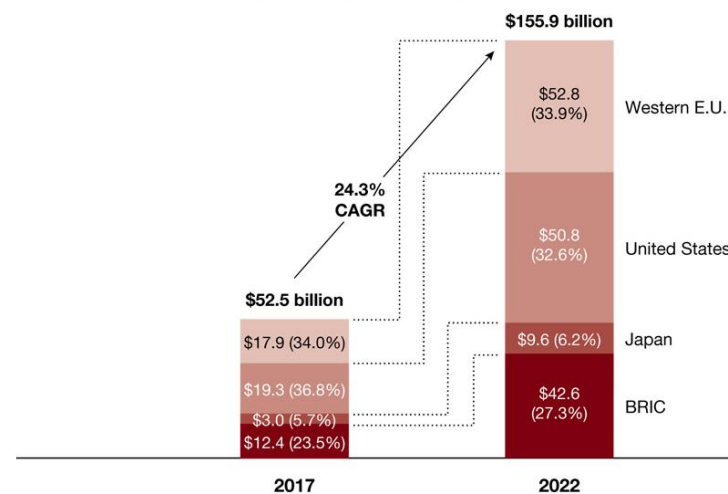
April 26, 2017

WIRED  
ANDY GREENBERG SECURITY 02.16.17 5:30 PM

### ANDROID PHONE HACKS COULD UNLOCK MILLIONS OF CARS

Exhibit 7

Connected car revenue potential, by region, 2017-22



INFOTAINMENT:  
WAY IN

UP TO 100 INTERCONNECTED  
ELECTRONIC COMPONENTS



Ca' Foscari  
University  
of Venice

# Ariane 5 (June, 4th 1996)



[https://www.youtube.com/watch?v=gp\\_D8r-2hww](https://www.youtube.com/watch?v=gp_D8r-2hww)



# Ariane 5, what happened?

- Ariane 5 flight 501
  - Exploded 37" after the launch
  - Self-destructed because of an overflow
  - Cost: 370M\$
- Overflow due to a data conversion from a 64-bit floating point to 16-bit signed integer
  - [https://en.wikipedia.org/wiki/Ariane\\_flight\\_V88](https://en.wikipedia.org/wiki/Ariane_flight_V88)
- Bug discovered by a static analyzer
  - Polyspace
  - <https://en.wikipedia.org/wiki/Polyspace>

# Airbus issues major A320 recall, threatening global flight disruption

By Tim Hepher, Allison Lampert, Dan Catchpole and David Shepardson

November 29, 2025 12:32 AM GMT+1 · Updated November 29, 2025



Aa



## Summary

## Companies

- Airbus orders immediate repairs to 6,000 A320 jets
- Repairs needed due to flight-control software issue
- Incident linked to solar flares affecting flight controls
- Some jets may also need hardware change, sources say

<https://www.reuters.com/business/aerospace-defense/airbus-issues-major-a320-recall-after-flight-control-incident-2025-11-28/>



sw bug

cause a damage

Takeout:

1. Software contains bugs
2. Bugs might cause a damage

That means:

1. There is no shame on writing bugged software
2. There is shame in not taking all countermeasures to avoid bugs!

# Program/Code analysis

We will focus on

- static
- automatic

program analyzers (SPA)

Main questions:

1. Who should use SPA and when?
2. Why using SPA?
3. Which SPA? How to evaluate?

*Program analysis is the process of **automatically** analyzing the **behavior** of computer programs regarding a **property** such as correctness, robustness, safety and liveness*

*Program analysis can be performed **without executing** the program (**static** program analysis), during **runtime** (**dynamic** program analysis) or in a combination of both.*



# Static Program Analyzers (SPA)

## Input and output

### Properties

- SQL Injection
- NullPointerExceptions
- Privacy leaks
- ...

### Software



**SPA**



[Injection] possible SQL-injection

[Injection] possible XSS-injection

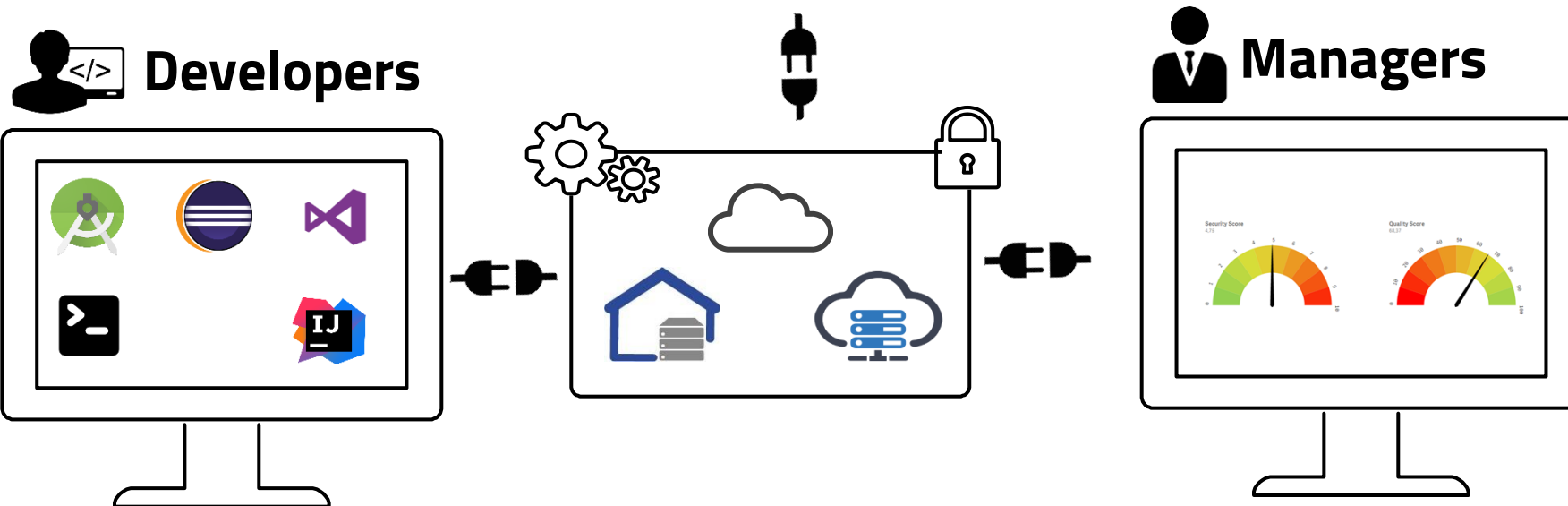
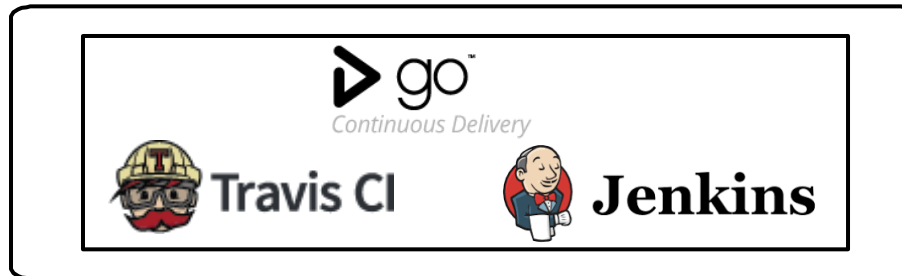
[BasicNullness] is the receiver of  
"getProperty" non-null?



# Who

## Developers, project managers, CTOs

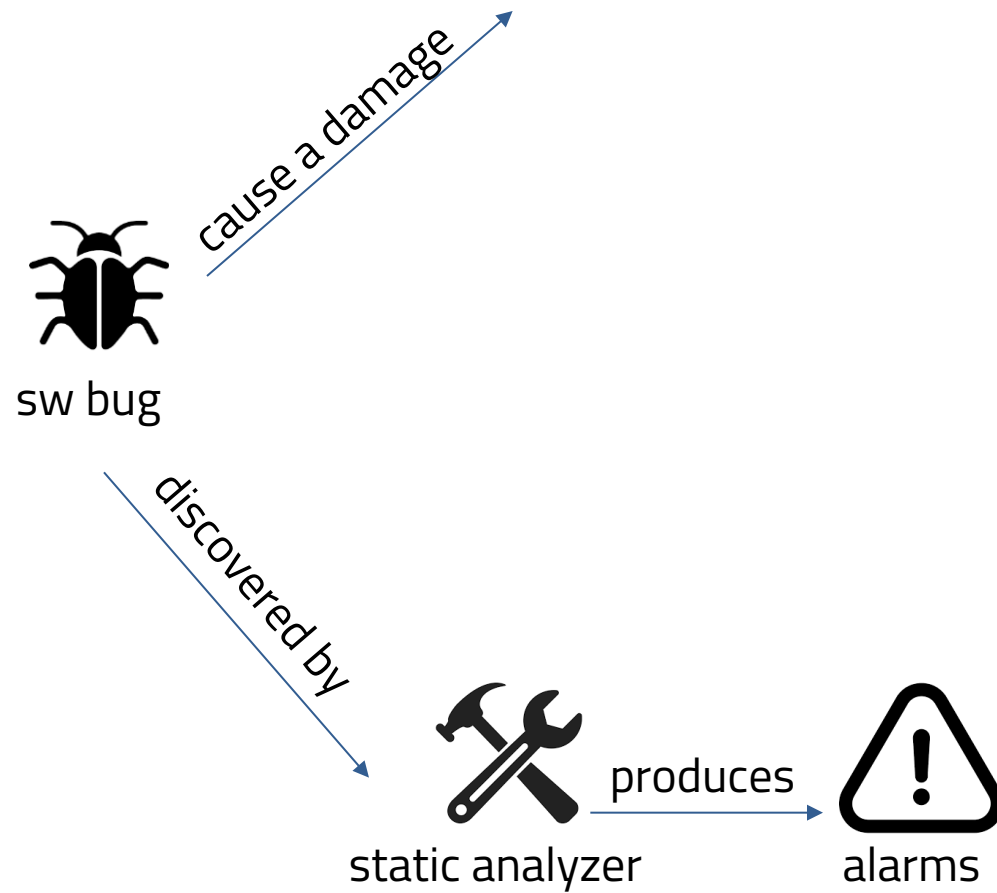
### Continuous integration





# SPAs' features

- Syntactic vs. semantic
- Source code vs. bytecode
- Sound vs. unsound
- Evaluation
- Difficult to compare them
- Syntactic: efficient, rough
- Semantic: slow, precise
- Bytecode: analyze 3rd party software, need to compile the whole system
- Sound: identify all possible (and more) bugs, security vulnerabilities, privacy leaks
- Various proposals:
  - OWASP Benchmark
  - NIST SAMATE
  - Static Analysis Technologies Evaluation Criteria





Ca' Foscari  
University  
of Venice

# Mitre Corporation

## <https://www.mitre.org/>



*The MITRE Corporation's mission-driven team is dedicated to solving problems for a safer world. We are a **not-for-profit company** that operates multiple **federally funded research and development centers**.*

- Born in 1958 as a private, not-for-profit company
- Operate federally funded research and development centers:
  - Scientific research and analysis
  - Development and acquisition
  - Systems engineering and integration
- Independent research program
  - explores new and expanded uses of technologies



- Dictionary of publicly disclosed cybersecurity vulnerabilities and exposures
  - free to search, use, and incorporate into products and services, per the terms of use.
- Feeds the U.S. National Vulnerability Database (NVD)
- CVE is:
  - One identifier for one vulnerability or exposure
  - One standardized description for each vulnerability or exposure
  - How disparate databases and tools can "speak" the same language
  - A basis for evaluation among services, tools, and databases



# Vulnerability definition by CVE

"A **weakness** in the computational logic (e.g., code) found in **software and hardware** components that, when exploited, results in a **negative** impact to **confidentiality, integrity, or availability**. Mitigation of the vulnerabilities in this context typically involves **coding changes**, but could also include **specification changes** or even **specification deprecations** (e.g., removal of affected protocols or functionality in their entirety)."



Ca' Foscari  
University  
of Venice

# National Institute of Standards and Technology

<https://www.nist.gov/>



*To **promote U.S. innovation and industrial competitiveness** by advancing **measurement** science, **standards**, and **technology** in ways that enhance economic security and improve our quality of life.*

- Measurement standards laboratory
- Non-regulatory agency
- United States Department of Commerce
- Formed on March, 3rd 1901 (!)
- 1901-1988: National Bureau of Standards
  - provide standard weights and measures
  - national physical laboratory for the United States



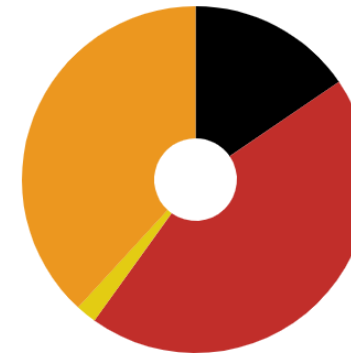
# NIST National Vulnerability DB

<https://nvd.nist.gov/>

- U.S. government repository of standards based vulnerability management data

| Time Period | New CVEs Received by NVD | New CVEs Analyzed by NVD | Modified CVEs Received by NVD | Modified CVEs Re-analyzed by NVD |
|-------------|--------------------------|--------------------------|-------------------------------|----------------------------------|
| Today       | 63                       | 1                        | 0                             | 0                                |
| This Week   | 167                      | 98                       | 52                            | 1                                |
| This Month  | 238                      | 225                      | 158                           | 1                                |
| Last Month  | 1371                     | 1065                     | 1030                          | 6                                |
| This Year   | 2874                     | 2363                     | 4173                          | 35                               |

CVSS V3 Score Distribution



| Severity | Number of Vulns |
|----------|-----------------|
| CRITICAL | 3550            |
| HIGH     | 10200           |
| MEDIUM   | 8764            |
| LOW      | 445             |



Ca' Foscari  
University  
of Venice

# CVSS



<https://www.first.org/cvss/>

The Common Vulnerability Scoring System (CVSS) provides a way to capture the principal characteristics of a vulnerability and produce a **numerical score reflecting its severity**. The numerical score can then be translated into a **qualitative representation** (such as low, medium, high, and critical) to help organizations properly **assess and prioritize their vulnerability management processes**.

- Free and open industry standard for assessing the severity of computer system security vulnerabilities
- Maintained by FIRST

FIRST is an **international confederation** of trusted **computer incident response teams** who cooperatively handle computer security incidents and promote incident prevention programs.



- The challenge:
  - Software acquirers want assurance that software products are reviewed for known types of security flaws
  - There are no nomenclature, taxonomies, or standards to define the capabilities and coverage of these tools
- The solution
  - A list of common software security weaknesses
  - It encompasses a large portion of the CVE List
- MITRE began working on the issue of categorizing software weaknesses as early 1999 when it launched the CVE List



# Common Weakness Enumeration

## The lingua franca

CWE is a:

- list of software weakness types
- common language for describing software security weaknesses
- measuring stick for software security tools
- common baseline standard for weakness identification

Each weakness type is identified by a unique number

### CWE 89

**Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')**

### CWE 79

**Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')**

**[Almost] all existing SPAs are CWE compatible**  
**<http://cwe.mitre.org/compatible>**



# Log4J vulnerability

- <https://logging.apache.org/log4j/2.x/>
  - Any software MUST do logging!
- But then a security vulnerability there might have some impact
  - <https://en.wikipedia.org/wiki/Log4Shell>
  - <https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2021-44228>
  - <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>

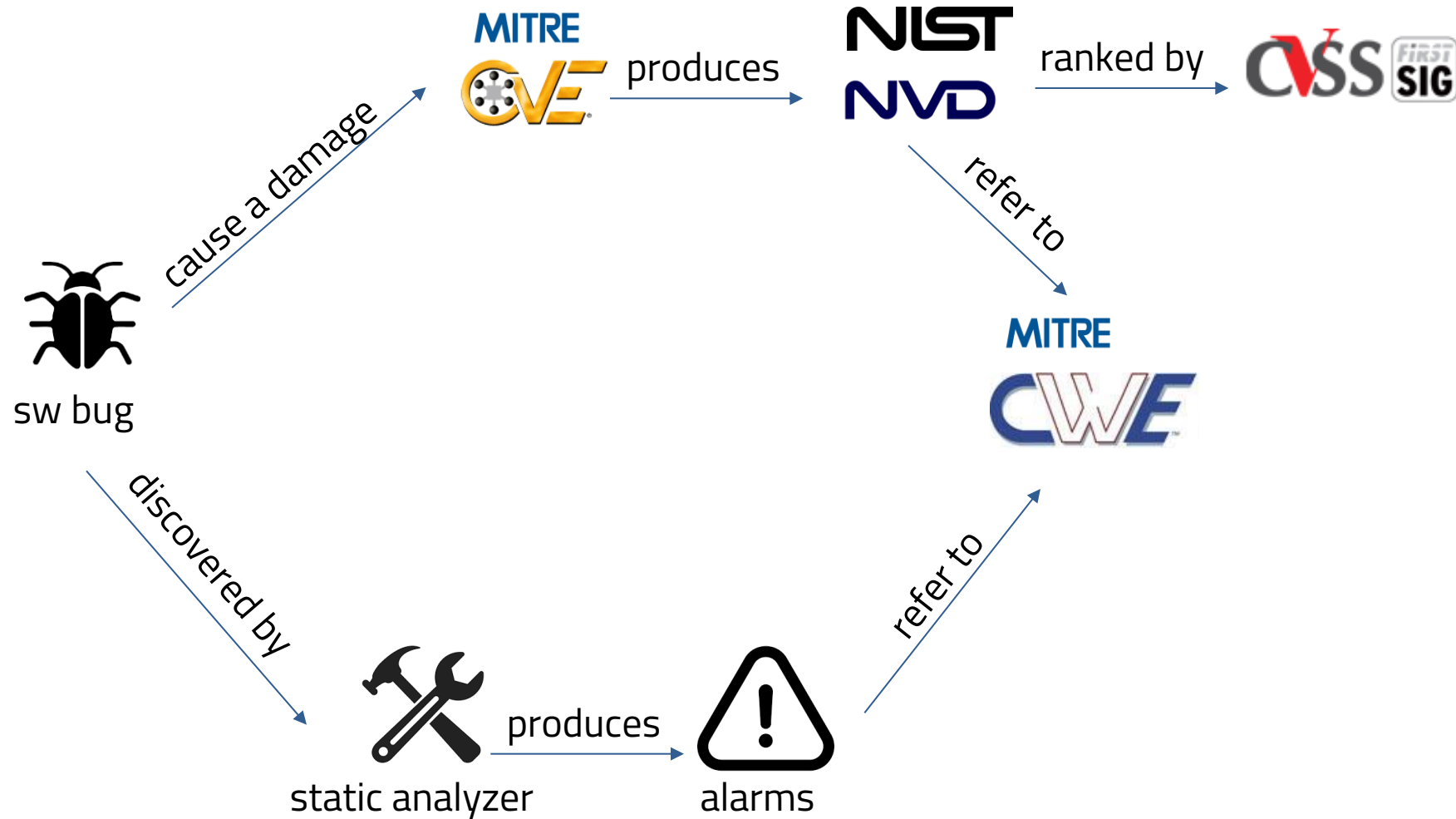
*The exploit allows hackers to gain **control of vulnerable devices** using Java (...) In the days following the vulnerability's disclosure, Check Point observed millions of attacks being initiated by hackers, with some researchers observing a rate of over one hundred attacks per minute that ultimately resulted with attempted attacks on over 40% of business networks internationally.*



# Heartbleed bug

## A notable example

- <http://heartbleed.com/>
- Discovered on April 1<sup>st</sup>/3<sup>rd</sup> 2014 by Google and Codenomicon
  - <https://en.wikipedia.org/wiki/Codenomicon>
- Fixed on April 7<sup>th</sup>
- Quite some noise in the news
- Few exploits:
  - April 8<sup>th</sup>: steal of SSN of 900 Canadian tax payers
  - Hijacking of account of <https://www.mumsnet.com/>
  - Steal of 4.5 million patient records of U.S. hospitals
- <https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160>





Ca' Foscari  
University  
of Venice

# CWE/SANS TOP 25 Most Dangerous SW Errors

<http://cwe.mitre.org/top25>

| Rank | ID             | Name                                                                                       | Score | CVEs in KEV | Rank Change vs. 2022 |
|------|----------------|--------------------------------------------------------------------------------------------|-------|-------------|----------------------|
| 1    | <u>CWE-787</u> | Out-of-bounds Write                                                                        | 63.72 | 70          | 0                    |
| 2    | <u>CWE-79</u>  | Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')       | 45.54 | 4           | 0                    |
| 3    | <u>CWE-89</u>  | Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')       | 34.27 | 6           | 0                    |
| 4    | <u>CWE-416</u> | Use After Free                                                                             | 16.71 | 44          | +3                   |
| 5    | <u>CWE-78</u>  | Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') | 15.65 | 23          | +1                   |
| 6    | <u>CWE-20</u>  | Improper Input Validation                                                                  | 15.50 | 35          | -2                   |
| 7    | <u>CWE-125</u> | Out-of-bounds Read                                                                         | 14.60 | 2           | -2                   |
| 8    | <u>CWE-22</u>  | Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')             | 14.11 | 16          | 0                    |
| 9    | <u>CWE-352</u> | Cross-Site Request Forgery (CSRF)                                                          | 11.73 | 0           | 0                    |
| 10   | <u>CWE-434</u> | Unrestricted Upload of File with Dangerous Type                                            | 10.41 | 5           | 0                    |
| 11   | <u>CWE-862</u> | Missing Authorization                                                                      | 6.90  | 0           | +5                   |



Ca' Foscari  
University  
of Venice

# Open Web Application Security Project

<https://www.owasp.org/>

- An online community that produces freely-available articles, methodologies, documentation, tools, and technologies in the field of web application security
- History:
  - Born on September 9<sup>th</sup>, 2001
  - OWASP foundation established in 2004
  - OWASP Europe VZW registered in Belgium in 2011
- 368 projects
  - [https://www.owasp.org/index.php/Category:OWASP Project#tab=Project Inventory](https://www.owasp.org/index.php/Category:OWASP_Project#tab=Project_Inventory)





Ca' Foscari  
University  
of Venice

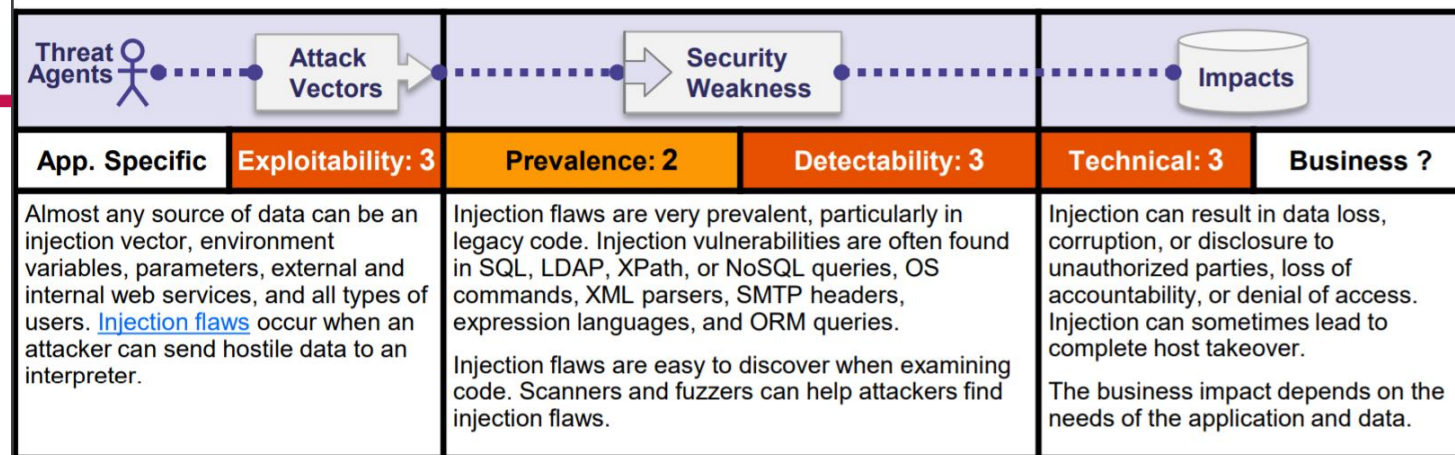
# OWASP Top 10 2017

[https://www.owasp.org/index.php/Top\\_10-2017\\_Top\\_10](https://www.owasp.org/index.php/Top_10-2017_Top_10)

**"The OWASP Top 10 provides a list of the 10 Most Critical Web Application Security Risks"**

De-facto standard about most dangerous SW vulnerabilities

| ID | Description                                 |
|----|---------------------------------------------|
| 1  | Injection                                   |
| 2  | Broken Authentication                       |
| 3  | Sensitive Data Exposure                     |
| 4  | XML External Entities (XXE)                 |
| 5  | Broken Access Control                       |
| 6  | Security Misconfiguration                   |
| 7  | Cross-Site Scripting (XSS)                  |
| 8  | Insecure Deserialization                    |
| 9  | Using Components with Known Vulnerabilities |
| 10 | Insufficient Logging&Monitoring             |



## Example Attack Scenarios

**Scenario #1:** An application uses untrusted data in the construction of the following [vulnerable](#) SQL call:

**String query = "SELECT \* FROM accounts WHERE custID=" + request.getParameter("id") + "";**

**Scenario #2:** Similarly, an application's blind trust in frameworks may result in queries that are still vulnerable, (e.g. Hibernate Query Language (HQL)):

**Query HQLQuery = session.createQuery("FROM accounts WHERE custID=" + request.getParameter("id") + "");**

In both cases, the attacker modifies the 'id' parameter value in their browser to send: ' or '1'='1. For example:

**http://example.com/app/accountView?id=' or '1'='1**

This changes the meaning of both queries to return all the records from the accounts table. More dangerous attacks could modify or delete data, or even invoke stored procedures.

## References

### OWASP

- [OWASP Proactive Controls: Parameterize Queries](#)
- [OWASP ASVS: V5 Input Validation and Encoding](#)
- [OWASP Testing Guide: SQL Injection, Command Injection, ORM injection](#)
- [OWASP Cheat Sheet: Injection Prevention](#)
- [OWASP Cheat Sheet: SQL Injection Prevention](#)
- [OWASP Cheat Sheet: Injection Prevention in Java](#)
- [OWASP Cheat Sheet: Query Parameterization](#)
- [OWASP Automated Threats to Web Applications – OAT-014](#)

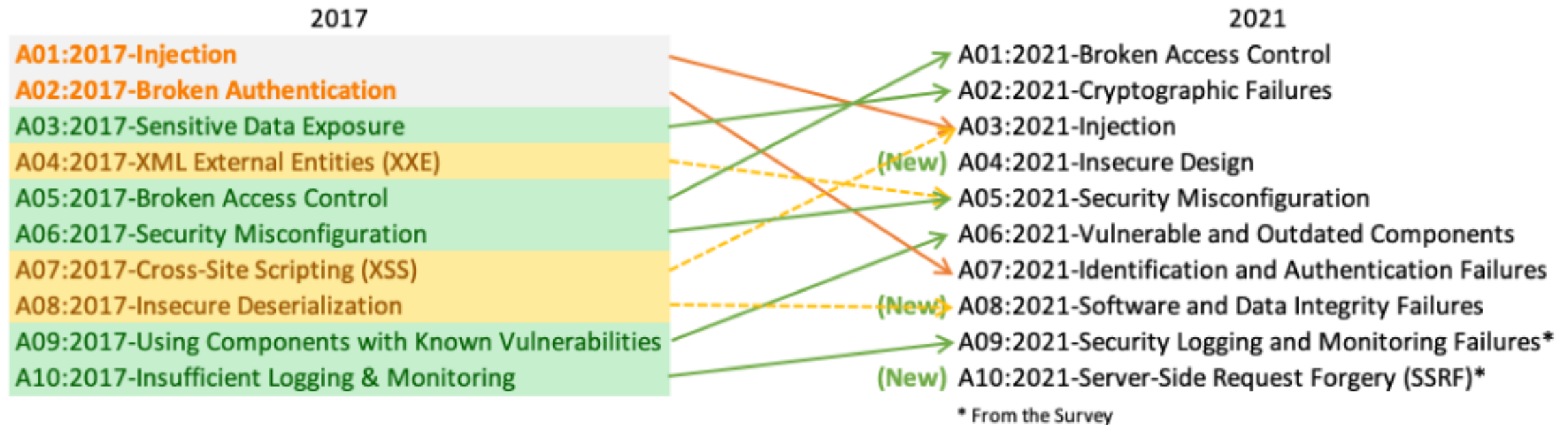
### External

- [CWE-77: Command Injection](#)
- [CWE-89: SQL Injection](#)
- [CWE-564: Hibernate Injection](#)
- [CWE-917: Expression Language Injection](#)
- [PortSwigger: Server-side template injection](#)



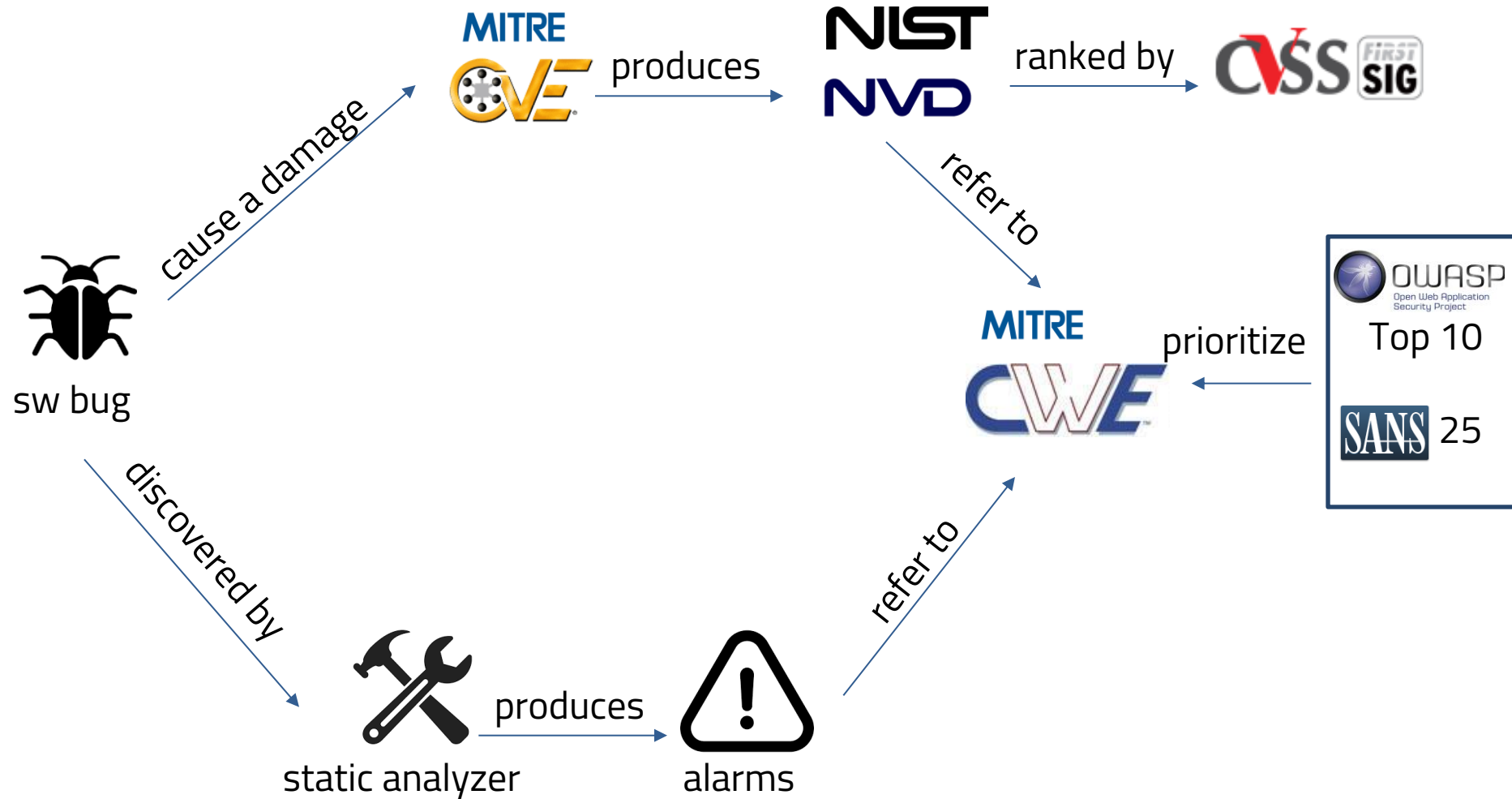
# From 2017 to 2021

<https://owasp.org/www-project-top-ten/>



- **A03:2021-Injection** slides down to the third position. 94% of the applications were tested for some form of injection, and the 33 CWEs mapped into this category have the second most occurrences in applications. Cross-site Scripting is now part of this category in this edition.

[https://owasp.org/Top10/A03\\_2021-Injection/](https://owasp.org/Top10/A03_2021-Injection/)

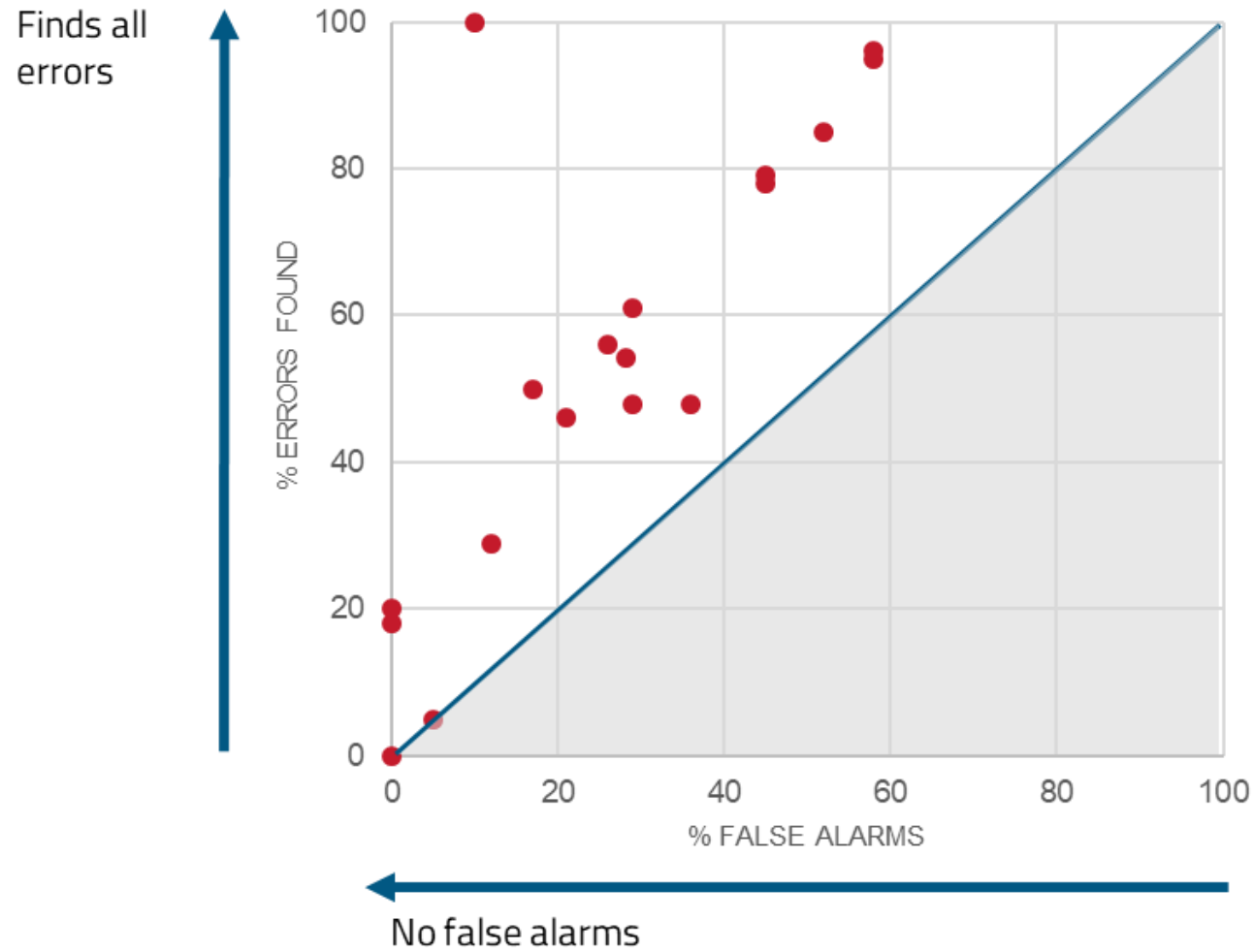




# The OWASP Benchmark

<https://www.owasp.org/index.php/Benchmark>

- Free and open test suite
  - **Evaluate** the speed, coverage, and accuracy of static analyzers
  - Fully runnable and exploitable
  - **Security** issues like injection, XSS, weak cryptographic algorithms, ...
- About 2700 **Java** test cases
  - evaluate program analyzers on OWASP Top 10 vulnerabilities
- Mapped to **CWE** vulnerability ids
  - **True** positive: vulnerabilities detected by the tool (coverage)
  - **False** positive: warning that is not a real vulnerability (precision)





# NIST Software Assurance Metrics And Tool Evaluation

The SAMATE project is dedicated to improving software assurance by developing methods to enable **software tool evaluations**, **measuring the effectiveness** of tools and techniques, and **identifying gaps** in tools and methods. The scope of the SAMATE project is **broad**: ranging from operating systems to firewalls, SCADA to web applications, **source code security analyzers** to correct-by-construction methods.

- Define the Bug Framework
  - <https://samate.nist.gov/BF/>
- Juliet test suite for C/C++ and Java
  - <https://samate.nist.gov/SARD/testsuite.php>
- Software Assurance Reference Dataset Project
  - <https://samate.nist.gov/SARD/>



Ca' Foscari  
University  
of Venice

# Web Application Security Consortium (WASC)



<http://www.webappsec.org/>

*The Web Application Security Consortium (WASC) is 501c3 non profit made up of an **international** group of experts, industry practitioners, and organizational representatives who produce **open source** and widely agreed upon **best-practice security standards for the World Wide Web**.*

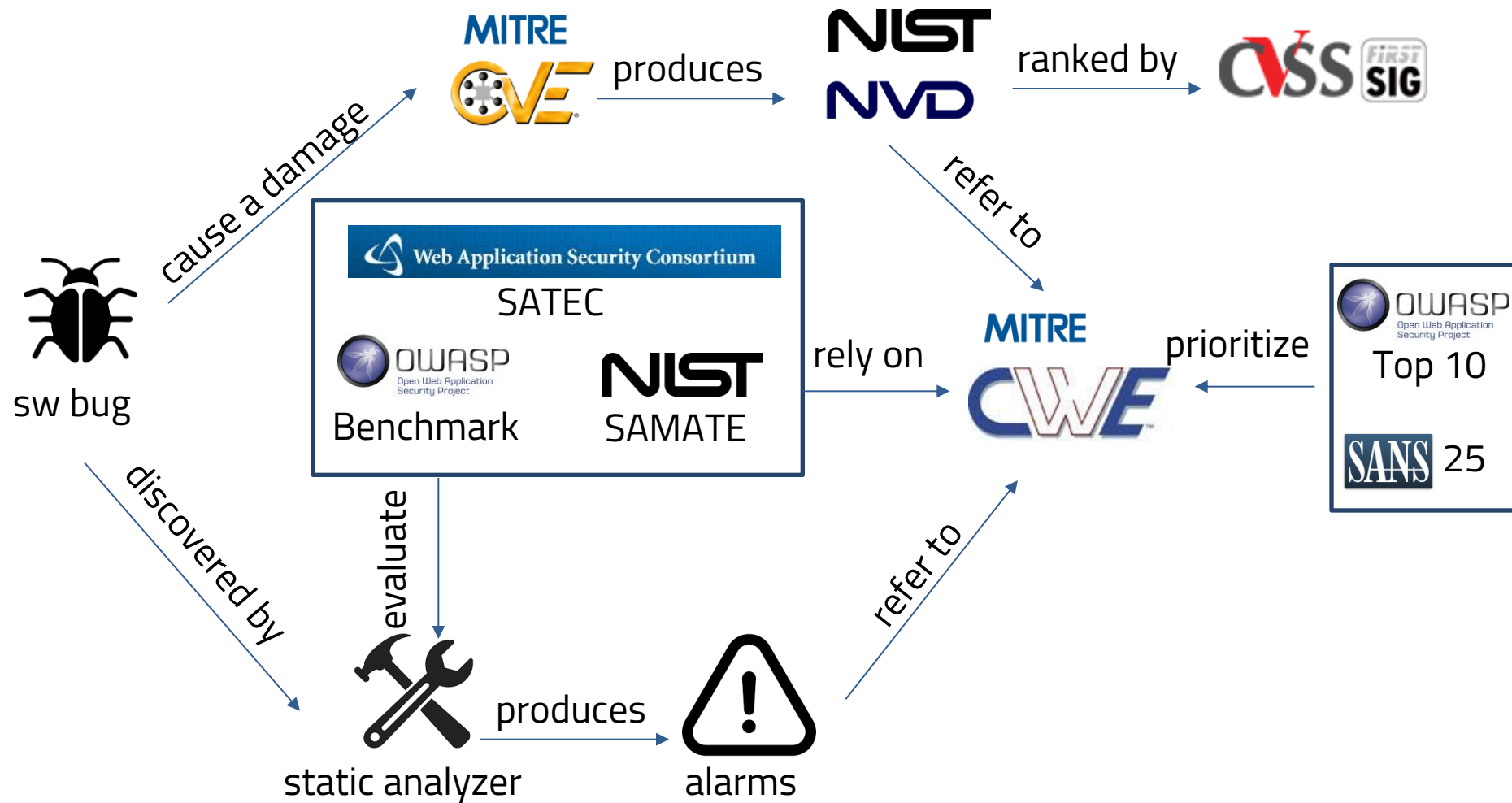
- Exchange of ideas, several industry projects
- Releases technical information, contributed articles, security guidelines, etc..
- Focused to: businesses, educational institutions, governments, application developers, security professionals, and software vendors



# WASC Static Analysis Technologies Evaluation Criteria

The goal of the SATEC project is to create a **vendor-neutral** set of criteria to help guide application security professionals during the process of **acquiring a static code analysis technology** that is intended to be used during **source-code driven security programs**.

- Guidelines to evaluate commercial static analyzers
  - Based on “functionalities” and not on test cases
  - More about coverage than precision
- Spreadsheet to evaluate different tools





Ca' Foscari  
University  
of Venice

# Standards

International standards and regulations require compliance for assuring software quality for several industries.

Recommended techniques include static analysis.



## **ISO DIS 26262-6**

Road vehicles –  
functional safety



## **RTCA DO-178C**

Airborne systems and  
equipment certification



## **IEC 62304:2006**

Medical device software





Ca' Foscari  
University  
of Venice

# Privacy

## GDPR: European General Data Protection Regulation

**1**

Implement appropriate technical and organisational measures in order to ensure data protection

**2**

Report data breach no later than  
**72 hours**

**3**

Fines up to **4%** of global revenue or  
**€ 20 Million**

## COMPLIANCE BY MAY 2018

# Focal points



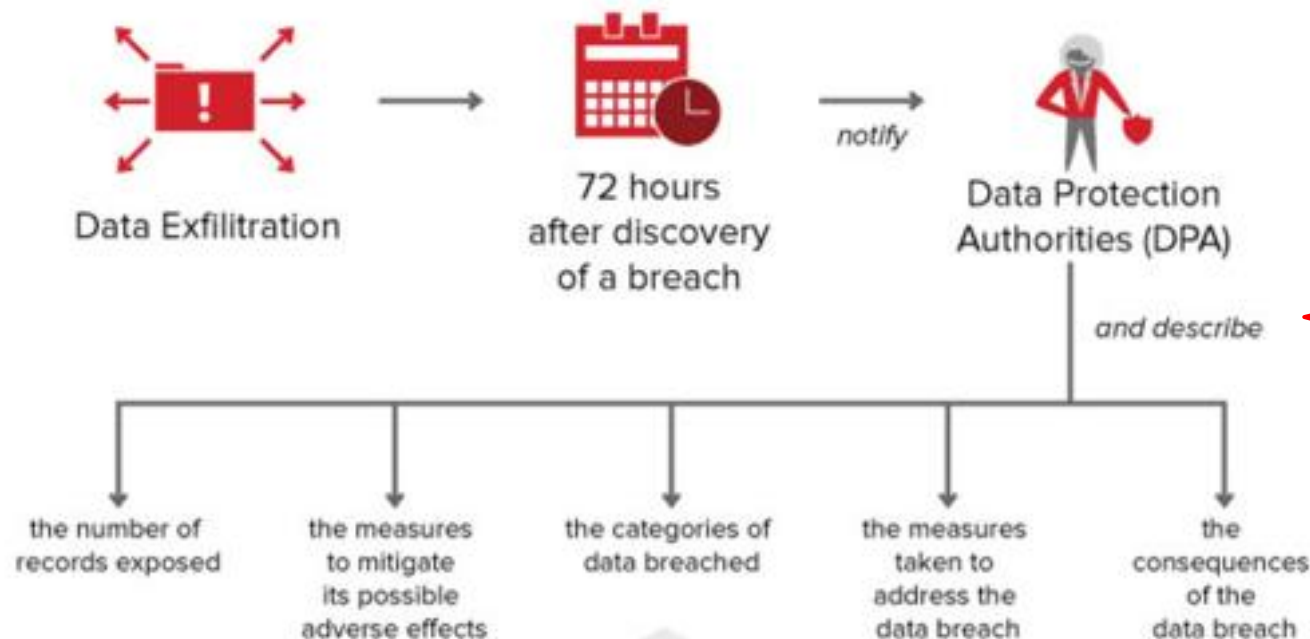
|                                |                                                                                                                                             |                                |                                                                                                                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ARTICLE<br><b>15</b>           | Grants EU citizens the <b>RIGHT OF ACCESS</b> which requires companies to detail what personal data is being processed and how upon request | ARTICLE<br><b>17</b>           | Grants EU citizens the <b>RIGHT TO BE FORGOTTEN AND TO DATA ERASURE</b> which requires companies to stop processing and delete personal data upon request             |
| ARTICLE<br><b>20</b>           | Grants EU citizens the <b>RIGHT TO DATA PORTABILITY</b> to enable citizens to transfer personal data between companies upon request         | ARTICLES<br><b>25 &amp; 32</b> | Require companies to implement <b>REASONABLE DATA PROTECTION MEASURES</b> to protect EU citizens' personal data and privacy by design                                 |
| ARTICLES<br><b>33 &amp; 34</b> | Require companies to <b>REPORT DATA BREACHES TO SUPERVISORY AUTHORITIES AND INDIVIDUALS</b> affected by a breach within 72 hours            | ARTICLE<br><b>35</b>           | Requires companies to perform <b>DATA PROTECTION IMPACT ASSESSMENTS</b> to identify risks to EU citizen data and outline measures to ensure those risks are addressed |
| ARTICLE<br><b>37</b>           | Requires certain companies to <b>APPOINT DATA PROTECTION OFFICERS</b> to oversee data security strategy and GDPR compliance                 | ARTICLE<br><b>50</b>           | <b>EXTENDS DATA PROTECTION REQUIREMENTS TO INTERNATIONAL COMPANIES</b> that collect or process EU citizens' personal data                                             |

# In case of Data Breach: support forensics

## BREACH RESPONSE

### WHAT'S A BREACH?

A breach of security leading to "accidental or unlawful destruction, loss, alteration, unauthorized disclosure of, or access to personal data".



Sanctions up to **4%** of global revenue, or **€ 20 Mn**



In case of incident use Julia to trace data, identify vulnerable points and **gain awareness** of all the data possibly compromised.

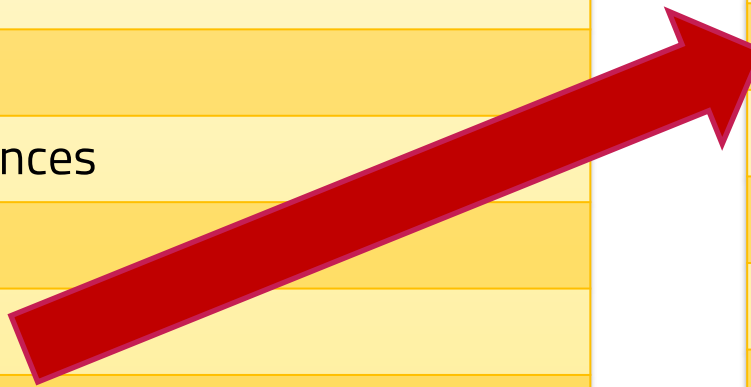




Ca' Foscari  
University  
of Venice

# OWASP Top 10 from 2013 to 2017

| ID | 2013                                         |
|----|----------------------------------------------|
| 1  | Injection                                    |
| 2  | Broken Authentication and Session Management |
| 3  | Cross-Site Scripting (XSS)                   |
| 4  | Insecure Direct Object References            |
| 5  | Security Misconfiguration                    |
| 6  | Sensitive Data Exposure                      |
| 7  | Missing Function Level Access Control        |
| 8  | Cross-Site Request Forgery (CSRF)            |
| 9  | Using Components with Known Vulnerabilities  |
| 10 | Unvalidated Redirects and Forwards           |



| ID | 2017                                        |
|----|---------------------------------------------|
| 1  | Injection                                   |
| 2  | Broken Authentication                       |
| 3  | Sensitive Data Exposure                     |
| 4  | XML External Entities (XXE)                 |
| 5  | Broken Access Control                       |
| 6  | Security Misconfiguration                   |
| 7  | Cross-Site Scripting (XSS)                  |
| 8  | Insecure Deserialization                    |
| 9  | Using Components with Known Vulnerabilities |
| 10 | Insufficient Logging&Monitoring             |



# Sensitive Data Exposure From 2013 to 2017



## Am I Vulnerable to Data Exposure?

The first thing you have to determine is which data is sensitive enough to require extra protection. For example, passwords, credit card numbers, health records, and personal information should be protected. For all such data:



## Is the Application Vulnerable?

The first thing is to determine the protection needs of data in transit and at rest. For example, passwords, credit card numbers, health records, personal information and business secrets require extra protection, particularly if that data falls under privacy laws, e.g. EU's General Data Protection Regulation (GDPR), or regulations, e.g. financial data protection such as PCI Data Security Standard (PCI DSS). For all such data:



# Sensitive Data Exposure

## Am I Vulnerable?

- Determine protection needs
- Consider if falls under regulation
- Any data transmitted in clear text?
- Stored in clear text?
- Weak cryptographic algorithms?

## How do I Prevent This?

- Classify data
- Review privacy laws
- Encrypt all sensitive data
- Encrypt all data in transit
- Ensure strong cryptographic algorithms

**Static program analyzers can help you checking that**

- **All sensitive data is encrypted**
- **All data in transit is encrypted**
- **Weak cryptographic algorithms are not used**



Ca' Foscari  
University  
of Venice

# Sensitive Data Exposure

## Current coverage



- CWE Entry 310 on Cryptographic Issues
- CWE Entry 312 on Cleartext Storage of Sensitive Information
- CWE Entry 319 on Cleartext Transmission of Sensitive Information
- CWE Entry 326 on Weak Encryption

### Julia

- CWE-319: Injection
- CWE-327, 328:  
Cryptography
  - Sons of CWE-326

### SonarQube

- CWE-326, 327, 328:
  - Rules S2089, S2278, S2245, S2277, S2258, S2257, S2070



# Sensitive Data Exposure From 2013 to 2017



- [CWE Entry 310 on Cryptographic Issues](#)
- [CWE Entry 312 on Cleartext Storage of Sensitive Information](#)
- [CWE Entry 319 on Cleartext Transmission of Sensitive Information](#)
- [CWE Entry 326 on Weak Encryption](#)



[CWE-359 Exposure of Private Information \(Privacy Violation\)](#)  
[CWE-220 Exposure of sens. information through data queries](#)  
[CWE-310: Cryptographic Issues](#); [CWE-326: Weak Encryption](#)  
[CWE-312: Cleartext Storage of Sensitive Information](#)  
[CWE-319: Cleartext Transmission of Sensitive Information](#)



## Exposure of Private Information

**The software does not properly prevent private data (such as credit card numbers) from being accessed by actors who either (1) are not explicitly authorized to access the data or (2) do not have the implicit consent of the people to which the data is related.**

SPAs might be applied to

1. Comprehensively **identify** all accesses to personal data
2. **Track** its flow in the program
3. **Detect** where such data could be leaked

### Examples

#### Leakage

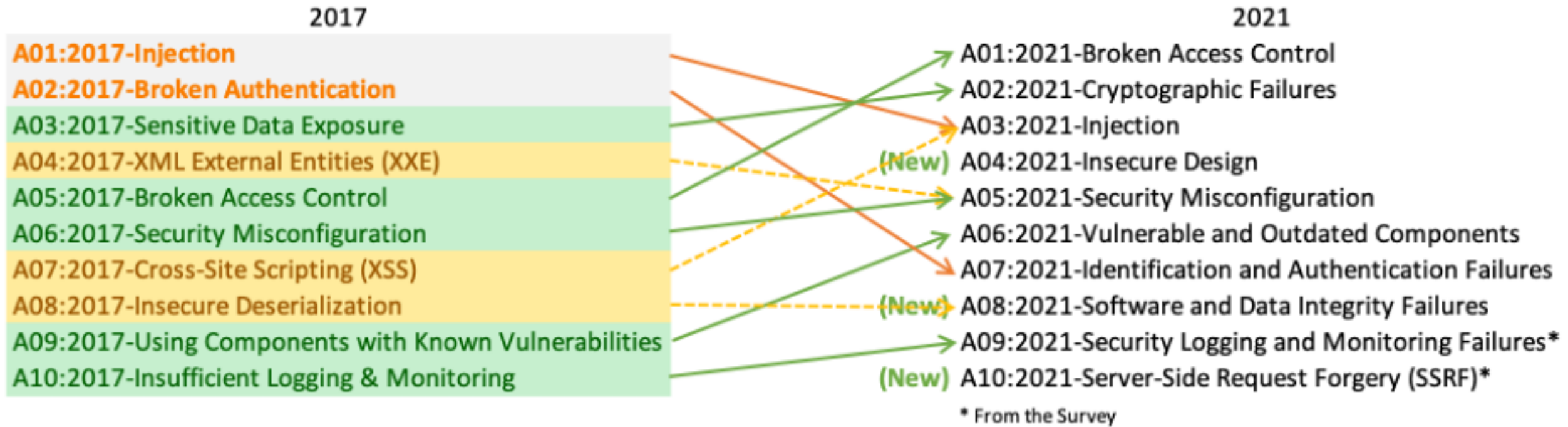
```
pass = getPassword();  
log.write(pass);
```

#### Sensible data not needed

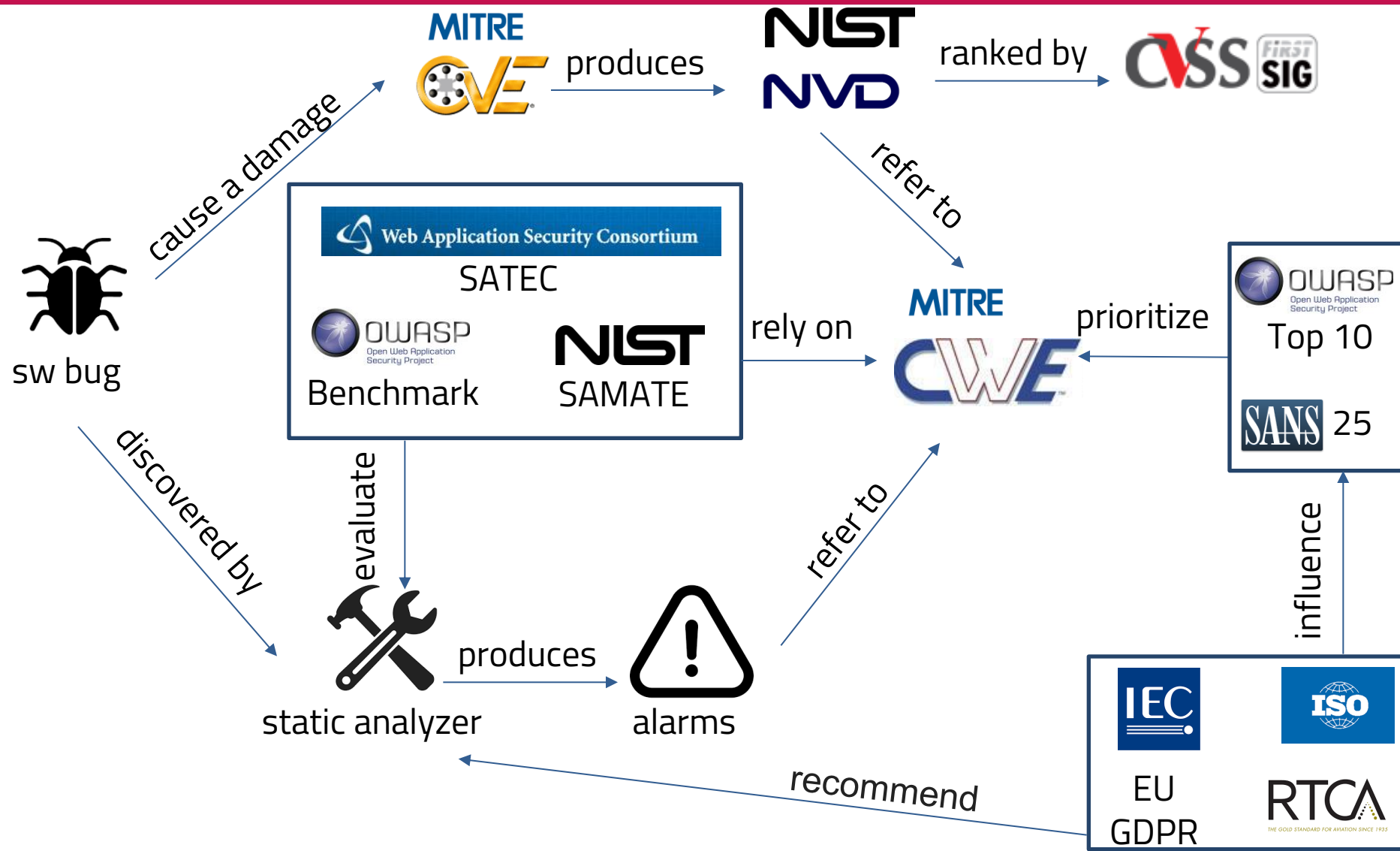
```
Permission ACCESS_FINE_LOCATION  
Location loc = getLastLocation();  
deriveStateFromCoords(loc);  
ACCESS_COARSE_LOCATION enough!
```



# From 2017 to 2021



Huge effort to collect data: <https://owasp.org/www-project-top-ten/#div-data> 2020  
Many more CWE ids... but 359 disappeared!





# OASIS Static Analysis Results Interchange Format (SARIF)



- [https://www.oasis-open.org/committees/tc\\_home.php?wg\\_abbrev=sarif](https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=sarif)
- Standard output format for static analysis tools
- Allow developers and teams to:
  - view, understand, interact with, and manage
  - the results produced by a variety all the tools that they use
- Support aggregation of the results of many tools
- Comprehensively capture the range of data produced by commonly used static analysis tools
- Capture information useful for assessing a project's compliance with corporate policy or conformance to certification standards.
- Current standard

